

Importing data

Responses to exercise questions

Put your name here

Reading (seemingly) non-tabular data

Fama-French industry definitions

Exercises

1. Follow the steps below to produce a data set where each column has the appropriate data type.
 - a. Go to the MSCI GICS website.
 - b. Get the link to the file under *historical GICS structures* that is “Effective until Sep 28, 2018”.
 - c. Use `pl.read_excel()` to read the linked file directly from the URL.
 - d. Identify any variables that need to be handled like `temp` in the Fama-French data set above and process accordingly.
 - e. Use `.forward_fill()` to fill in rows as necessary.
 - f. Make sure that each column has the appropriate data type.

Hints:

- a. You may find it helpful to look at the Excel file so you can see how the data are structured.
- b. `pl.read_excel()` requires an Excel engine package such as `fastexcel`.
- c. `pl.read_excel()` has options that can help you skip rows and assign column names.
- d. Your *final* column names should be `sector`, `sector_desc`, `ind_group`, `ind_group_desc`, `industry`, `industry_desc`, `sub_ind`, `sub_ind_desc`, and `sub_ind_details`.
- e. The following code snippets *might* be useful:
 - `df[df["sub_ind_details"].notna()]`
 - `df[cols] = df[cols].ffill()` (where `cols` is a list of columns to forward-fill)
 - `df.convert_dtypes()`

Extracting data from messy formats

Exercises

1. In the PDF, Ritter states “IPOs marked with an * also had international tranches (and probably some others, too).” Write code to remove this “*” when present and create an additional column `intl_tranche` that indicates an international tranche. (*Hint:* The `extract_groups()` function can be used here. In a first pass, you might use named capturing groups and `regex = r"^(?`

`P<company>.*?)(?P<intl_tranche>\\*?)$`". Can you see what the `?` in `.*?` is doing? There is some explanation here. Does the regex work without this `??`? What does `\\*?` match?

- Ritter defines *money left on the table* as “the difference between the closing price on the first day of trading and the offer price, multiplied by the number of shares sold.” Can you calculate this from the data provided? Is the calculated amount (`amount`) equal to the amount in `left_on_table` in each case? What explains the differences? (*Hints*: There will be more than one reason. You may find it helpful to calculate `ratio = amount / left_on_table` and to focus on differences of more than 1% with `.filter(abs(ratio - 1) > 0.01)`.)
- In words, what do each of the following regular expressions *match*? What do they capture?
 - `r"^\s+"`
 - `r"(.+)\s+"`
 - `r"([^\s]+)\s+"`
 - `r"([0-9]{1,2}/[0-9]{4})\s+"`
 - `r"([0-9,]+\s)+"`
- What happens if we parse `shares_offered` as floating-point numbers rather than integers?
- What happens if we apply `.str.extract_groups(regex)` directly to `pl.DataFrame({"line": list(pdf_lines)})`?
- The online appendix for “The Customer Knows Best: The Investment Value of Consumer Opinions” contains Table OA.1 entitled *List of firms with customer reviews on Amazon.com*. Using an approach similar to that we used for `ritter_data` above, create a data frame `huang_data` with columns `company_name`, `industry`, `start`, `end`, `months`, `reviews`.
 - `months` and `reviews` should be numerical values.
 - `start` and `end` should be dates (use the first day of the month if only the month and year is specified).
 - You can solve this by combining the partial regular expressions above into a single regular expression by joining Python raw strings, as we did for `ritter_data`.
- Using the following code, where the URL provides a sample of observations on mergers and acquisitions from SDC, create a first-pass import of the data.¹ What is the first issue you see when you look at the data in `ma_sdc`? (*Hint*: Look at the first few rows.) Adapt the code to address this issue. (*Hint*: You may have to experiment with different values for `skiprows` to get the right setting.)

```
url = ("https://gist.githubusercontent.com/iangow/"
      "eb7dfelcd0913821429bdf0566465d41/raw/"
      "358d60a4429f5747abc61f8acc026d335fc165f3/sap_sample.txt")

def url_to_file(url):
```

¹We messed with these data, so these cannot be used for research! But they are a realistic representation of an actual data set.

```

    resp = requests.get(url)
    resp.raise_for_status()
    return io.BytesIO(resp.content)

with url_to_file(url) as f:
    ma_sdc = read_fwf(
        f,
        widths=[13, 13, 40, 19, 37, 16, 17, 10, 10, 12, 12, 16, 21, 23, 10, 1000],
        names=
            ["date_announced", "date_effective", "tgt_name",
             "tgt_nation", "acq_name", "acq_nation",
             "status", "pct_of_shares_acq", "pct_owned_after_transaction",
             "acq_cusip", "tgt_cusip", "value_of_transaction_mil",
             "acq_prior_mktval", "tgt_prior_mktval",
             "acq_nation_code", "tgt_nation_code"],
    )
ma_sdc

```

8. Fix the variables `date_announced` and `date_effective` so that they have type date. (*Hint:* Use `parse_dates=["date_announced", "date_effective"]` and `date_format="%m/%d/%y".`)
9. What are the minimum and maximum values of `date_announced` and `date_effective`? What explains missing values (if any) here?
10. What do you observe about `acq_cusip` and `tgt_cusip`? Can you write some code to check that these variables have been read in correctly? (*Hint:* The `.str.len()` method might be useful here.)